

Note — Non-Obvious Design Choices

B22AI053 — Assignment 2

Manas Chechani

PART I: WHY I USED CHUNK-LEVEL LID INSTEAD OF PURE FRAME-LEVEL CLASSIFICATION

The assignment says frame-level LID, and I did build a frame-level classifier on top of wav2vec2 features with a multi-head attention layer. But here's the thing — when I was actually running it on the 10-minute segment, the raw frame-level predictions were noisy. A lot of frames near switch boundaries would flip back and forth between Hindi and English every few milliseconds, which looked terrible and gave wrong switch timestamps.

So what I ended up doing was I ran the frame-level model to get per-frame posteriors, but then I aggregated them over 30-second chunks for the final language label. For the switch timestamps, I used the frame posteriors with a median filter (window of ~50 frames, roughly 1 second) to smooth out the jitter before picking the actual transition point. This is why my switch precision stayed well within the 200ms requirement. I think this is more honest than reporting perfect frame-level accuracy but having garbage switch detection — the smoothing is the part that actually matters in practice.

PART II: THE IPA MAPPING WAS HARDER THAN I EXPECTED

I initially thought I could just use `epitran` for Hindi and `eng_to_ipa` for English and stitch them together. That works for monolingual text, but Hinglish broke it badly. Words like “motivate” appearing in otherwise Hindi sentences got transliterated with Devanagari rules (came out as something like “moe” instead of “motvet”).

My solution was to first use the LID labels from Part I to tag each word, and then route it to the right G2P backend — Hindi words through a custom Devanagari-to-IPA rule table I wrote (handling things like schwa deletion and nasalization that `epitran` misses), and English words through the CMU-based converter. For the 500-word parallel corpus for Maithili, I honestly could not find any existing MT system, so I manually mapped the most frequent content words from the transcript and padded it with technical terms from the syllabus. It's not a proper translation — more like a glossary-driven substitution — but that's the best you can do without a Maithili NMT model.

PART III: DTW PROSODY WARPING — THE TRICKY BIT WAS ALIGNMENT, NOT THE DTW ITSELF

Everyone in class probably used the same `dtw-python` or `librosa.sequence.dtw` call. That part is straightforward. The non-obvious decision I made was about *what* to align. I extracted F0 contours from both the professor's audio and my synthesized output using CREPE (because pYIN was giving me too many unvoiced gaps on the noisy lecture audio).

But the professor's lecture is ~600 seconds and my synthesized Maithili output has completely different duration. So before DTW, I had to segment both by utterance boundaries (using the 30s chunk timestamps from Part I), run DTW per chunk, and then concatenate. If you try to DTW the entire 10-minute F0 contour at once, it just finds a degenerate warping path that maps everything to a flat region. Per-chunk alignment gave much more sensible contours, and my MCD came out at 7.52, which passes the <8.0 threshold.

PART IV: LFCC BEAT CQCC BY A HUGE MARGIN — AND I THINK I KNOW WHY

My anti-spoofing classifier with LFCC features got an EER of 8.33%, but the same architecture with CQCC gave 41.67% — basically random. I think the reason is that my synthesized audio from VITS has a very specific spectral fingerprint in the linear frequency bands (probably related to the vocoder's upsampling artifacts), and LFCC captures that directly. CQCC uses a log-frequency scale which spreads those artifacts across bins and dilutes the signal.

Also, my training set was small (just my real voice samples vs. the cloned output), so the model needed clear, separable features to learn from — LFCC gave that, CQCC didn't. For the FGSM adversarial attack, the minimum epsilon that flipped Hindi→English was ~0.00064 with SNR of 59.9 dB, meaning the perturbation is completely inaudible. This tells me my LID model is fairly brittle to targeted gradient attacks even though it has high clean accuracy — something I'd want to fix with adversarial training if this were a production system.